

1. Overview

PDFSolid Conversion SDK is a high-performance library designed for extracting and transforming the data within your PDF files, such as text, images, tables, links, and annotations, into various file formats. Our Conversion SDK retains the original document layout and the properties of the file data, ensuring a seamless document conversion experience.

Effortlessly integrate the PDFSolid Conversion SDK into your Ruby projects in just a few steps, and enable the following file format conversions:

- Convert PDF to Word (.docx)
- Convert PDF to Excel (.xlsx)
- Convert PDF to PowerPoint (.pptx)
- Convert PDF to HTML (.html)
- Convert PDF to CSV (.csv)
- Convert PDF to Image (.png, .jpg, .jpeg, .jpeg2000, .bmp, .tiff, .tga, .gif, .webp)
- Convert PDF to Plain Text (.txt)
- Convert PDF to Rich Text Format (.rtf)
- Convert PDF to Searchable PDF (.pdf)
- Convert PDF to OFD (.ofd)
- Convert PDF to Structured Data (.json)
- Convert PDF to Markdown (.md)

Harness the power of PDFSolid Conversion SDK and take your document processing to the next level with intelligent tools designed for accuracy and efficiency. To enhance your format conversion results, PDFSolid offers AI-powered document tools with the following capabilities:

- Optical Character Recognition (OCR)
- Layout Analysis
- Table Recognition

1.1 Why PDFSolid Conversion SDK

- Mature Technology

With years of technology accumulation, we have established a complete mechanism of product iteration to offer a continuous guarantee for product competitiveness.

- Complete PDF and Format Conversion Functionalities

Our comprehensive features can meet diverse needs and are easy for our customers to use without training costs.

- High-quality Service

Professional service and technical support to quickly respond to users' feedback through onsite service or remote support like telephone, email, etc.

- Independent Intellectual Property Rights

Our technology is independent and compliant with ISO, helping enterprises conduct international business without considering copyright risks.

1.2 PDFSolid Conversion SDK for Ruby

The PDFSolid Conversion Ruby SDK wraps the native PDFSolid Conversion C/C++ runtime for Linux x86_64. It uses the Ruby standard library `Fiddle` to call the native C API and includes a small C bridge for native structures.

The Ruby call chain is:

```
Ruby
-> Fiddle
-> Ruby C bridge
-> PDFSolid Conversion native library
-> PDFSolid Conversion C++ SDK
```

The current Ruby SDK release package supports Linux x86_64.

1.3 License & Trial

The PDFSolid Conversion SDK is a commercial SDK that requires a license to grant developers the right to develop and distribute their applications. In development mode, each license is only valid for one device ID. PDFSolid provides flexible licensing models, please contact [our marketing team](#) for more information. However, even if you have a license, it is prohibited to distribute any documents, sample code, or source code of the PDFSolid Conversion SDK to any third parties.

If you do not have a License, please feel free to contact the [PDFSolid Team](#) to obtain a License to try the PDFSolid Conversion SDK.

2. Get Started

2.1 Requirements

Before starting, please make sure that you have already met the following prerequisites.

2.1.1 Get PDFSolid License Key

PDFSolid provides two types of license key: 30-day free trial license and commercial license.

How to Get Free Trial License

[Contact our sales team](#) and we will send you a 30-day free trial license for PDFSolid Conversion SDK.

How to Get Commercial License

PDFSolid Conversion SDK is a commercial SDK that requires a license for application release. Any documents, sample code, or source code distribution from the released package of PDFSolid to any third party is prohibited.

To get a commercial license for PDFSolid Conversion SDK, feel free to [contact our sales team](#).

For Ruby Conversion SDK, commercial licenses are verified through the SDK license verification API. In development mode, a license is bound to the developer device ID.

2.1.2 Download Conversion SDK

[Contact us](#) to obtain the PDFSolid Conversion SDK for Ruby.

2.1.3 System Requirements

Development Platform	System Requirements	Development Environment	Notice
Linux	Linux x86_64	Ruby 2.7 or higher	Samples have been tested on Ubuntu 20.04.

The Ruby SDK does not require third-party Ruby gems at runtime. It uses Ruby standard libraries, including `Fiddle`, `FileUtils`, `OptionParser`, and `Thread` in samples.

2.2 SDK Package Structure

You can [contact us](#) to get the PDFSolid Conversion SDK for Ruby package. The Linux x86_64 release package follows the same top-level structure as the C++ SDK package. From the package root directory, it contains the following files:

```
doc/
  README.md
  api_reference_ruby.html
  developer_guide_ruby.md
  html/
lib/
  pdfsolid_conversion.rb
  pdfsolid_conversion/
    callback.rb
    conversion.rb
    enums.rb
    error_code.rb
    library_manager.rb
    native.rb
    options.rb
    version.rb
  libpdfsolidconversionsdk.so
  libDocumentAI.so.4.0.0
  libonnxruntime.so.1.18.0
  libopencv_world.so.410
  libpdfsolidconversion_ruby_bridge.so
resource/
  models/
    documentai.model
samples/
  demo/
    demo.rb
  input_files/
    word.pdf
    excel.pdf
    powerpoint.pdf
  output_files/
  license.xml
```

```
legal.txt
release_notes.txt
```

The main directories are:

- `doc/`: API reference and developer guide.
- `lib/`: Ruby source files loaded by `require "pdfsolid_conversion"`, the native PDFSolid Conversion runtime libraries, and the Ruby bridge library.
- `resource/`: DocumentAI model resources.
- `samples/`: Ruby demo script, sample input files, license file, and output directory.
- `legal.txt`: Legal and copyright information.
- `release_notes.txt`: Release information.

2.3 Apply the License Key

If you do not have a license key, please check out [how to obtain a license key](#).

PDFSolid Conversion SDK currently supports offline authentication to verify license keys.

2.3.1 Copy the License Key

Accurately obtaining the license key is crucial for applying the license.

1. In the email you received, locate the XML file containing the license key.
2. Open the XML file, and determine the license type based on the `<type>` field. If `<type>online</type>` is present, it indicates an online license. If `<type>offline</type>` is present or if the field is absent, it indicates an offline license.
3. Copy the value located at the `LICENSE_KEY` position within the `<key>LICENSE_KEY</key>` field. This is your license key.

2.3.2 Apply the License Key

Use `PdfSolidConversion::LibraryManager.license_verify` before initializing the SDK and starting conversions.

```
require "pdfsolid_conversion"

license = "LICENSE_KEY"
device_id = "DEVICE_ID"
app_id = "com.example.application"

code = PdfSolidConversion::LibraryManager.license_verify(license, device_id,
app_id)
unless code == PdfSolidConversion::ErrorCode::SUCCESS
  raise "license verification failed: #{code}"
end

PdfSolidConversion::LibraryManager.initialize_sdk
```

You can also read the license information from environment variables in command-line tools:

```
license = ENV.fetch("PDFSOLID_LICENSE")
device_id = ENV.fetch("PDFSOLID_DEVICE_ID", "")
app_id = ENV.fetch("PDFSOLID_APP_ID", "com.example.application")
```

The sample scripts also accept `--license`, `--device-id`, and `--app-id`. When these options are not provided, they read the same environment variables and fall back to the local demo values packaged in the samples.

2.4 How to Run a Demo

2.4.1 Linux

The Ruby SDK release package provides `samples/demo/demo.rb`. Run the commands from the package root directory that contains `doc/`, `lib/`, `resource/`, and `samples/`. When no input PDFs are specified, the demo converts the packaged sample PDFs in `samples/input_files` and writes output files to `samples/output_files`.

```
cd /path/to/linux-x86_64
ruby samples/demo/demo.rb
```

You can also pass custom inputs and output directory:

```
ruby samples/demo/demo.rb \
  --output /tmp/pdfsolid_ruby_demo_out \
  /path/to/input.pdf
```

You can pass license information through command-line options:

```
ruby samples/demo/demo.rb \
  --license /path/to/license.xml \
  --device-id DEVICE_ID \
  --app-id com.example.application \
  --output /tmp/pdfsolid_word_out \
  /path/to/input.pdf
```

Or through environment variables:

```
export PDFSOLID_LICENSE="/path/to/license.xml"
export PDFSOLID_DEVICE_ID="DEVICE_ID"
export PDFSOLID_APP_ID="com.example.application"
ruby samples/demo/demo.rb --output /tmp/pdfsolid_word_out /path/to/input.pdf
```

The demo defaults to the package resources when arguments are not provided:

- License: `samples/license.xml`, or `PDFSOLID_LICENSE` / `PDFSOLID_LICENSE_KEY`.
- DocumentAI model: `resource/models/documentai.model`, or `PDFSOLID_MODEL_PATH`.
- Input PDFs: `samples/input_files/word.pdf`, `samples/input_files/excel.pdf`, and `samples/input_files/powerpoint.pdf`.
- Output directory: `samples/output_files`.

The `demo.rb` sample also demonstrates multi-process PDF to Word conversion when explicit input PDFs are provided.

```
ruby samples/demo/demo.rb \  
  --license /path/to/license.xml \  
  --output /tmp/pdfsolid_ruby_demo_out \  
  --threads 2 \  
  samples/input_files/word.pdf \  
  samples/input_files/excel.pdf
```

To load a different DocumentAI model in the demo, pass `--model`:

```
ruby samples/demo/demo.rb \  
  --output /tmp/pdfsolid_ruby_demo_out \  
  --model /path/to/documentai.model \  
  /path/to/input.pdf
```

3. Conversion Guides

PDFSolid Conversion SDK allows developers to use a simple API to convert PDF to commonly used file formats such as Word, Excel, PowerPoint, HTML, CSV, PNG, JPEG, RTF, TXT, Searchable PDF, OFD, JSON, and Markdown. It provides a wide range of customized conversion options, such as whether to include images or annotations in PDF documents, whether to enable OCR, whether to enable layout analysis, and more.

All Ruby conversion APIs follow the same basic signature:

```
result = PdfSolidConversion::Conversion.start_pdf_to_word(  
  input_file_path,  
  password,  
  output_file_path,  
  options,  
  callback: callback  
)
```

`password` can be an empty string when the input PDF is not encrypted. Each conversion method returns an `ErrorCode` value. `ErrorCode::SUCCESS` means the conversion succeeded.

3.1 Initialize Library Resources

Overview

Initialize the necessary file and memory resources required by the PDFSolid Conversion SDK.

The Ruby SDK initializes with its packaged internal resource directory. You do not need to pass a resource path.

Notes

- You must verify the license and initialize SDK resources before calling any conversion interface.
- When using OCR, Layout Analysis, Table Recognition, PDF to Searchable PDF, or PDF to OFD, make sure the DocumentAI model is available and loaded when the feature requires it.

Example

```
PdfSolidConversion::LibraryManager.initialize_sdk
```

3.2 Set DocumentAI Model

Overview

Before using OCR, Layout Analysis, or Table Recognition, you need to set the DocumentAI model path first.

`set_document_ai_model` supports an optional `gpu_id` parameter used to specify the GPU device index for the AI model. When `gpu_id` is `-1`, GPU acceleration is disabled.

Set AI Model Instance Count

If you need to control the number of Layout Analysis and Table Recognition model instances, call `set_document_ai_model_count`.

Sample

```
model_path = "/path/to/documentai.model"
PdfSolidConversion::LibraryManager.set_document_ai_model_count(1, 1)
code = PdfSolidConversion::LibraryManager.set_document_ai_model(model_path, -1)
raise "set_document_ai_model failed: #{code}" unless code ==
PdfSolidConversion::ErrorCode::SUCCESS
```

3.3 Get Conversion Progress

PDFSolid Conversion SDK obtains the conversion progress through callback functions. The following example demonstrates how to get conversion progress while performing a PDF to Word task.

```
callback = PdfSolidConversion::ConvertCallback.new(
  progress: ->(current_page, total_page) {
    puts "progress: #{current_page}/#{total_page}"
  }
)
```

Pass the callback through the `callback:` keyword argument:

```
result = PdfSolidConversion::Conversion.start_pdf_to_word(  
  input_file_path,  
  "",  
  output_file_path,  
  PdfSolidConversion::ConvertOptions.new,  
  callback: callback  
)
```

3.4 Cancel Conversion Task

PDFSolid Conversion SDK supports interrupting an ongoing conversion task. Return `true` from the cancel callback to cancel the current task, or `false` to continue.

```
cancel_requested = false  
  
callback = PdfSolidConversion::ConvertCallback.new(  
  cancel: -> { cancel_requested }  
)
```

3.5 Select Page Range for Conversion

PDFSolid Conversion SDK supports converting a specified page range. When an empty string is passed, all pages will be converted. If the page range exceeds one page, you can also choose to enable the `output_document_per_page` option to output each PDF page as a separate file.

```
options = PdfSolidConversion::ConvertOptions.new  
options.page_ranges = "1-3,5,7-9"  
options.output_document_per_page = false
```

3.6 Conversion Options: Contain Image & Annotation

Overview

In the process of converting PDF documents into various formats, PDFSolid Conversion SDK offers options to determine whether images are included in the generated document and whether annotations from the PDF file are retained.

In the Ruby SDK, use `contain_image` and `contain_annotation` on `ConvertOptions`.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.contain_image = true
options.contain_annotation = true
options.enable_ai_layout = true
options.enable_ocr = false

result = PdfSolidConversion::Conversion.start_pdf_to_word(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.7 Page Layout Mode

In certain formats, the page layout mode plays a key role in the quality of the converted document. PDFSolid Conversion SDK supports two layout modes: Flow Layout and Box Layout.

- **Flow Layout:** Uses paragraph indentations, columns, and tab positions to adjust the content. This layout is suitable for editable documents.
- **Box Layout:** Places each element based on the original PDF coordinate system. This layout is suitable for documents requiring precise reproduction.

Page layout modes are commonly used for PDF to Word and PDF to HTML.

```
options = PdfSolidConversion::ConvertOptions.new
options.page_layout_mode = PdfSolidConversion::PageLayoutMode::FLOW

result = PdfSolidConversion::Conversion.start_pdf_to_word(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.8 OCR

Overview

OCR (Optical Character Recognition) is the process of converting images of typed, handwritten, or printed text into machine-encoded text.

OCR is commonly used for text recognition and extraction from scanned PDF files, photographs of documents, scene photos, invoices, receipts, and other image-based documents.

The following features support OCR:

- PDF to Word
- PDF to Excel
- PDF to PowerPoint
- PDF to HTML
- PDF to RTF
- PDF to TXT
- PDF to Searchable PDF

- PDF to OFD
- Extract PDF to JSON
- Extract PDF to Markdown

Set OCR Language

Use `languages` to specify OCR languages. The value is an array of `OCRLanguage` constants.

```
options = PdfSolidConversion::ConvertOptions.new
options.enable_ocr = true
options.languages = [
  PdfSolidConversion::OCRLanguage::ENGLISH,
  PdfSolidConversion::OCRLanguage::CHINESE
]
```

OCR Options

Use `ocr_option` to control OCR processing scope.

```
options.ocr_option = PdfSolidConversion::OCROption::ALL
```

Preserve Page Background

When OCR is enabled, use `contain_page_background_image` to control whether page background images are preserved.

```
options.contain_page_background_image = true
```

Sample

```
PdfSolidConversion::LibraryManager.set_document_ai_model("/path/to/documentai.model", -1)

options = PdfSolidConversion::ConvertOptions.new
options.enable_ocr = true
options.languages = [PdfSolidConversion::OCRLanguage::ENGLISH]
options.ocr_option = PdfSolidConversion::OCROption::ALL

result = PdfSolidConversion::Conversion.start_pdf_to_word(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.9 Layout Analysis

Overview

Layout Analysis detects the layout structure of a PDF page and helps improve output quality for complex pages.

Use `enable_ai_layout` to enable or disable AI layout analysis.

Sample

```
PdfSolidConversion::LibraryManager.set_document_ai_model("/path/to/documentai.model", -1)

options = PdfSolidConversion::ConvertOptions.new
options.enable_ai_layout = true

result = PdfSolidConversion::Conversion.start_pdf_to_word(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.10 Table Recognition

Overview

Table Recognition reconstructs the internal structure of tables detected during layout analysis, including rows, columns, merged cells, and cell boundaries, so that the converted document preserves the original tabular semantics instead of producing a flat grid of text fragments.

Use `enable_ai_table_recognition` to enable table recognition.

Sample

```
PdfSolidConversion::LibraryManager.set_document_ai_model("/path/to/documentai.model", -1)

options = PdfSolidConversion::ConvertOptions.new
options.enable_ai_table_recognition = true

result = PdfSolidConversion::Conversion.start_pdf_to_excel(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.11 Use Custom AI Models via Callbacks

Overview

C++ SDK v4.1.0 and later provides callback pairs for integrating custom OCR, layout analysis, and table recognition engines. The current Ruby SDK does not expose these callback registration APIs.

Ruby applications can use the built-in DocumentAI model through `set_document_ai_model`. If a custom AI engine is required, integrate it at the native C/C++ SDK layer and expose a Ruby wrapper only after the native callback lifecycle is fully controlled.

Notice

- Do not call undocumented native callback functions directly through `Fiddle`.
- Keep model ownership and callback lifetime in native code when adding a Ruby wrapper for custom AI callbacks.

3.12 Output Font Option

Overview

Some output formats support specifying a preferred output font. Use `font_name` in `ConvertOptions`.

Supported Formats

- PDF to Word
- PDF to Excel
- PDF to PowerPoint
- PDF to HTML
- PDF to RTF

Example

```
options = PdfSolidConversion::ConvertOptions.new
options.font_name = "Arial"
```

3.13 Convert PDF to Word

Overview

Converting PDF to Word is an operation that converts a PDF file into an editable Word file. By converting PDF to Word, you can edit, modify, insert, or delete text and images, and adjust layout and properties.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
result = PdfSolidConversion::Conversion.start_pdf_to_word(
  input_file_path,
  "",
  output_file_path,
  options
)
```

Convert Formulas to Images

When a document contains complex formulas and you want to preserve visual consistency in the output document, enable `formula_to_image`.

```
options = PdfSolidConversion::ConvertOptions.new
options.formula_to_image = true
```

3.14 Convert PDF to Excel

Overview

PDFSolid Conversion SDK supports converting PDF documents to Microsoft Excel format (.xlsx). By extracting, parsing, and importing data from PDF into Excel, users can further edit, analyze, or share Excel files.

Set the Content Options for Excel

When converting PDF files to Excel files, the following options directly affect content written to the Excel file.

- `excel_all_content`: whether the converted XLSX file contains all content in the PDF.
- `excel_csv_format`: whether to save table data in CSV format.
- `excel_worksheet_option`: worksheet layout option.

Option	Description
<code>ExcelWorksheetOption::FOR_TABLE</code>	Create one sheet per table.
<code>ExcelWorksheetOption::FOR_PAGE</code>	Create one sheet per page.
<code>ExcelWorksheetOption::FOR_DOCUMENT</code>	Create one sheet for the entire document.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.excel_worksheet_option =
  PdfSolidConversion::ExcelWorksheetOption::FOR_TABLE

result = PdfSolidConversion::Conversion.start_pdf_to_excel(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.15 Convert PDF to PowerPoint

Overview

PDFSolid Conversion SDK provides the function of converting PDF files to PowerPoint files and restoring the layout and format of the original document.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
result = PdfSolidConversion::Conversion.start_pdf_to_ppt(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.16 Convert PDF to HTML

Overview

PDFSolid Conversion SDK provides the PDF to HTML function, which can convert PDF files to HTML files while maintaining the layout and format of the original document.

HTML Options

Option	Description
<code>HtmlOption::SINGLE_PAGE</code>	Convert the entire PDF into a single HTML file.
<code>HtmlOption::SINGLE_PAGE_WITH_BOOKMARK</code>	Convert the PDF into a single HTML file with a bookmark navigation bar.
<code>HtmlOption::MULTI_PAGE</code>	Convert the PDF into multiple HTML files, one file per page.
<code>HtmlOption::MULTI_PAGE_WITH_BOOKMARK</code>	Convert the PDF into multiple HTML files with an outline page for navigation.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.html_option = PdfSolidConversion::HtmlOption::SINGLE_PAGE

result = PdfSolidConversion::Conversion.start_pdf_to_html(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.17 Convert PDF to CSV

Overview

PDFSolid Conversion SDK supports converting PDF documents to CSV (Comma-Separated Values). In the Ruby SDK, CSV output is controlled by Excel conversion options.

Set Whether to Automatically Create Folders

When multiple CSV files may be output, use `auto_create_folder` to control whether folders are automatically created.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.excel_csv_format = true
options.auto_create_folder = true

result = PdfSolidConversion::Conversion.start_pdf_to_excel(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.18 Convert PDF to Image

Overview

PDFSolid Conversion SDK provides an API for converting PDFs to images.

Setting Image Formats

Supported image formats include JPG, JPEG, JPEG2000, PNG, BMP, TIFF, TGA, GIF, and WEBP.

```
options = PdfSolidConversion::ConvertOptions.new
options.image_type = PdfSolidConversion::ImageType::PNG
```

Setting Image Color Modes

Supported image color modes include Color, Gray, and Binary.

```
options.image_color_mode = PdfSolidConversion::ImageColorMode::COLOR
```

Setting Image Scaling

The SDK supports setting image scaling. The default scaling is `1.0`.

```
options.image_scaling = 2.0
```

Enhancing Image Path Display

Use `image_path_enhance` to enhance the display effect of paths within the PDF page.

```
options.image_path_enhance = true
```

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.image_type = PdfSolidConversion::ImageType::PNG

result = PdfSolidConversion::Conversion.start_pdf_to_image(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.19 Convert PDF to RTF

Overview

RTF is a popular text format that can retain text format and style information and can be read by many text editors.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
result = PdfSolidConversion::Conversion.start_pdf_to_rtf(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.20 Convert PDF to TXT

Overview

When you need to extract text content from a PDF file for data analysis, text mining, or information retrieval, use PDF to TXT conversion.

Preserving Table Format

Use `txt_table_format` to preserve table format when writing the TXT file.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.txt_table_format = true

result = PdfSolidConversion::Conversion.start_pdf_to_txt(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.21 Convert PDF to Searchable PDF

Overview

Searchable PDF conversion adds a hidden text layer to an image-based PDF such as a scanned document using OCR.

Set Transparent Text Layer

Use `transparent_text` to control whether the text layer is transparent.

Sample

```
PdfSolidConversion::LibraryManager.set_document_ai_model("/path/to/documentai.model", -1)

options = PdfSolidConversion::ConvertOptions.new
options.enable_ocr = true
options.transparent_text = true
options.languages = [PdfSolidConversion::OCRLanguage::ENGLISH]

result = PdfSolidConversion::Conversion.start_pdf_to_searchable_pdf(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.22 Convert PDF to OFD

Overview

PDFSolid Conversion SDK supports converting PDF documents to OFD documents. OFD conversion also supports OCR, page background preservation, and transparent text layer options.

Sample

```
PdfSolidConversion::LibraryManager.set_document_ai_model("/path/to/documentai.mo
del", -1)

options = PdfSolidConversion::ConvertOptions.new
options.enable_ocr = true
options.languages = [PdfSolidConversion::OCRLanguage::ENGLISH]

result = PdfSolidConversion::Conversion.start_pdf_to_ofd(
  input_file_path,
  "",
  output_file_path,
  options
)
```

3.22 Releasing Library Resources

Overview

Release the files and memory resources occupied by PDFSolid Conversion SDK.

Notice

- After calling this interface to release library resources, the PDFSolid Conversion SDK will no longer function properly and must be initialized again before further conversion.
- When you only want to release resources occupied by the AI model, call `release_document_ai_model` instead of `release`.

Sample

```
PdfSolidConversion::LibraryManager.release_document_ai_model
PdfSolidConversion::LibraryManager.release
```

4 Data Extraction Guide

Unleash the power of data with PDFSolid Conversion SDK's data extraction capabilities to detect, recognize, analyze, and extract PDF text, images, tables, and more.

4.1 Extract PDF To JSON

Overview

Extract text, tables, and images from PDF documents to a JSON file.

Standard Table and Non-standard Table

Commonly, tables can be divided into two categories:

- Standard table: The table border and inner lines are complete and clear.
- Non-standard table: Tables lack borders or clear inner lines and require table recognition to recover structure.

Table Extraction Option

PDFSolid Conversion SDK supports the option `json_contain_table`. When enabled, table content is extracted from PDFs together with table structure; otherwise, table content is treated as regular text.

Notice

Without enabling AI layout analysis or OCR options, tables in the original PDF may not be extracted with high precision. It is recommended to enable AI layout analysis, OCR, or table recognition when extracting complex tables.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.json_contain_table = true
options.enable_ai_layout = true
options.enable_ai_table_recognition = true

result = PdfSolidConversion::Conversion.start_pdf_to_json(
  input_file_path,
  "",
  output_file_path,
  options
)
```

4.2 Extract PDF To Markdown

Overview

Extract text, tables, and images from PDF documents to a Markdown file.

Sample

```
options = PdfSolidConversion::ConvertOptions.new
options.enable_ai_layout = true
options.contain_image = true

result = PdfSolidConversion::Conversion.start_pdf_to_markdown(
  input_file_path,
  "",
  output_file_path,
  options
)
```

5. Support

5.1 FAQ

For frequently asked questions about license, device ID, platform support, and SDK integration, please visit the [PDFSolid FAQ](#).

5.2 Contact Us

PDFSolid is committed to providing high-quality technical support. If you encounter problems when integrating the Ruby SDK, please contact us.

Contact Information

- Website: <https://www.PDFSolid.com>
- Contact Sales: <https://www.PDFSolid.com/contact-sales>
- Email: support@PDFSolid.com